Are Self-Evolving Agent Skills Learning Skills?

Anonymous submission

Abstract

We introduce a lifecycle audit for determining whether score gains from self-evolving agent skills represent retained skill improvement. A post-revision score alone is insufficient because optimizer variation, validation selection, execution randomness, and verifier choice can improve the endpoint without creating a durable artifact effect. On 198 task variants, SkillEvolver yields a 14.90-point paired gain over no skill and outperforms SPARK and CoEvoSkills, including on the subset where all three supply artifacts. Across SkillRevise, SkillHone, and SkillOpt, we combine frozen parent-to-final deployment, controlled task variation, feedback and selection interventions, artifact-component mediation, and output-locked verifier counterfactuals. The audit separates five evidential claims: selected gain, retained artifact effect, bounded transfer, evolution-pathway dependence, and measurement dependence. This claim-grounded framework lets self-evolving systems report what their experiments establish without equating a better benchmark score with reusable skill learning.

Introduction

Persistent agent skills offer an appealing route to continual improvement. Instead of updating model parameters, an agent can revise an external artifact that stores reusable instructions, scripts, references, templates, or other supporting resources. Recent systems generate, repair, select, and retain such artifacts from execution traces and evaluator feedback, often reporting higher post-evolution benchmark scores as evidence that the agent has learned a better skill (Li and Hu 2026; Liu et al. 2026b; Yang et al. 2026; Xia et al. 2026).

A higher score is not, by itself, an identified skill effect. The selected run may be unusually favorable; the optimizer may produce variable candidates; a validation gate may retain a narrow patch; repeated execution may eventually succeed without a durable behavioral change; and a verifier may reward only part of the intended behavior. These possibilities do not imply that self-evolution is ineffective. They imply that an endpoint score alone cannot locate where the improvement resides.

We therefore ask two connected questions. When self-evolution raises the benchmark score, does the revised skill itself retain a behavioral improvement under fresh deployment and controlled task variation? Regardless of whether it does, which parts of the evolution-and-evaluation lifecycle produced the observed gain? We study these questions in SkillRevise, SkillHone, and SkillOpt, which represent trace-conditioned revision, history-based iterative honing, and validation-gated bounded editing. They form a mechanism-stratified sample of frozen-agent, feedback-driven systems with persistent external skill artifacts, rather than an exhaustive ranking of self-evolution methods.

We first evaluate SkillEvolver, SPARK, and CoEvoSkills on standard tasks and controlled variants, then use artifact content and execution traces to examine how successful procedures adapt to changed inputs and requirements.

Our audit follows the lifecycle from feedback to revision, selection or adoption, frozen artifact deployment, and measurement. We separate historical success from fresh retention, final-skill utility from actual parent-to-final revision effects, and same-task retention from controlled transfer. We then intervene on feedback, optimizer sampling, gate provenance, artifact components, and verifier choice. A short information-integrity check bounds direct answer or oracle exposure as an alternative explanation; it is not the paper's organizing hypothesis.

The resulting picture is deliberately more qualified than either *self-evolution works* or *self-evolution only exploits the evaluator*. Revised artifacts can cause real behavioral improvements, but the demonstrated effects are often narrow. Single-run or selected gains need not survive fresh deployment. Useful traces and feedback must be converted into an effective edit, adopted into the persistent artifact, and retained behaviorally. Finally, a better score can reflect compatibility with a measurement surface rather than broader task behavior.

Our central contribution is a claim-grounded lifecycle audit: each reported gain is matched to the intervention required to support it and to the strongest conclusion that intervention permits. We make three contributions:

- **Identification framework.** We separate a reported gain from retention, behavioral scope, evolution-and-selection pathway, and measurement dependence.
- **Lifecycle audit.** We apply matched interventions across three representative skill-evolution mechanisms without turning heterogeneous studies into a method leaderboard.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

自进化代理技能是否在学习技能?

匿名提交

摘要

我们介绍了一种生命周期审计方法，用于判断自进化代理技能带来的分数增益是否代表保留的技能改进。仅凭修订后的分数是不够的，因为优化器变异、验证选择、执行随机性和验证器选择可能会在不创建持久性工件效应的情况下提高终点分数。在198个任务变体上，SkillEvolver比无技能高出14.90分，并且优于SPARK和CoEvoSkills，包括在所有三个都提供工件的那部分子集上。在SkillRevise、SkillHone和SkillOpt中，我们结合了冻结的父代到最终部署、受控任务变异、反馈和选择干预、工件组件中介以及输出锁定验证器反事实。该审计将五个证据性主张分离出来：选择增益、保留的工件效应、有界迁移、进化路径依赖和测量依赖。这种基于主张的框架允许自进化系统报告其实验所证明的内容，而不将更好的基准分数等同于可重用的技能学习。

引言

持久的代理技能为持续改进提供了一条有吸引力的途径。代理无需更新模型参数，而是可以修订存储可重用指令、脚本、参考、模板或其他支持资源的的外部工件。最近的系统从执行轨迹和评估器反馈中生成、修复、选择和保留此类工件，通常报告更高的进化后基准分数，以此证明代理学习到了更好的技能（Li和Hu 2026；Liu等人 2026b；Yang等人 2026；Xia等人 2026）。

更高的分数本身并不能证明是识别出的技能效果。选定的运行可能异常有利；优化器可能产生不同的候选；验证门可能保留一个狭窄的补丁；重复执行最终可能在没有持久行为改变的情况下成功；验证器可能只奖励预期行为的一部分。这些可能性并不意味着自我进化是无效的。它们意味着仅凭终点分数无法定位改进所在之处。

因此，我们提出两个相互关联的问题。当自我进化提高基准分数时，修订后的技能是否在新的部署和控制任务变化下本身仍能保持行为改进？无论它是否能够做到，进化与评估生命周期中哪些部分产生了观察到的收益？我们在SkillRevise、SkillHone和SkillOpt中研究这些问题，它们分别代表轨迹条件下的修订、基于历史的迭代精炼和验证门控的边界编辑。它们形成一个机制分层的样本，包含冻结代理、反馈驱动的系统以及持久的技能外部工件，而不是自我进化方法的详尽排名。

我们首先在标准任务和控制变体上评估 SkillEvolver、SPARK 和 CoEvoSkills，然后使用工件内容和执行轨迹来检验成功的程序如何适应变化的输入和需求。

我们的审计遵循从反馈到修订的生命周期，包括选择或采纳、冻结工件部署以及测量。我们将历史成功与新鲜留存区分开，最终技能效用与实际父级到最终修订效果区分开，以及同一任务留存与受控迁移区分开。然后我们对反馈、优化器采样、门控溯源、工件组件和验证器选择进行干预。一次信息完整性检查将直接答案或或然器暴露作为替代解释，而不是论文的组织假设。

由此产生的图景比 自我进化有效 或 自我进化仅利用评估者 都更谨慎。修订后的工件可以导致真实的行为改进，但展示的效果往往是狭隘的。单次运行或选定的收益不必在新鲜部署中存活。有用的轨迹和反馈必须转化为有效的编辑，被采纳到持久的工件中，并在行为上得以保留。最后，更好的分数可能反映与测量表面的兼容性，而不是更广泛的任务行为。

我们的核心贡献是一种基于主张的生命周期审计：每个报告的收益都与支持它的干预措施以及干预措施允许的最强结论相匹配。我们做出了三项贡献：

- **识别框架**。 我们将报告的收益与保留、行为范围、进化与选择路径以及测量依赖分开。
- **生命周期审计**。我们针对三种具有代表性的技能进化机制应用匹配的干预措施，同时避免将异构研究变成方法排行榜。

这是一份仅供评审目的的匿名提交。严禁传播、引用或公开分享本稿件。若稿件被接受，版权和出版细节将出现在最终版本中。

- **Reporting protocol.** We specify the lineage, contrasts, and artifacts required to support each claim level, from selected gain to bounded transfer.

Problem Formulation

Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback that may include a verifier result, a gold answer, a diagnosis, or evaluator feedback. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

The Identification Question

Most evaluations observe:

$$y_{r+1} > y_r. \tag{4}$$

This inequality describes two evaluations; it does not identify what the persistent skill learned. The revised artifact may encode a retained behavioral improvement, the selected score may depend on optimizer or execution variation, or the output may better match the verifier's measurement surface. Infrastructure-invalid observations, such as a verifier that never reaches its semantic assertions, are neither successes nor failure mechanisms and must be repaired or excluded.

Accordingly, we separate two questions:

1. **Effect existence and scope:** Does the revised skill retain a behavioral improvement under fresh deployment, and how far does that effect extend under controlled task variation?
2. **Effect attribution:** Which feedback, revision, selection, artifact, execution, and measurement mechanisms produced the observed score change?

The following experiments address these questions in turn.

Experiments on SkillsBench

Experimental Setup

We evaluate SkillEvolver (Zhang et al. 2026a), SPARK (Zhou et al. 2026b), and CoEvoSkills (Zhang et al. 2026b) on SkillsBench (Li et al. 2026), with two runs per method. Each run pairs an execution using the generated or evolved skill with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{5}$$

We also count tasks with positive, negative, and zero differences.

Results across Two Runs

All three methods achieve positive mean paired gains in both runs (Table 1). SkillEvolver improves from 30.20 to 55.81 in run 1 and from 37.32 to 62.34 in run 2, yielding gains of 25.61 and 25.02 percentage points. The corresponding task counts are 27 improvements versus six regressions, and 26 improvements versus three regressions.

SPARK increases mean reward from 17.82 to 19.66 in run 1 and from 14.37 to 20.81 in run 2. Its paired gain rises from 1.84 to 6.45 points, with nine versus 13 improved tasks and eight versus five regressions. CoEvoSkills improves from 42.21 to 52.44 and from 23.54 to 33.31 on the aligned 85-task set. Its paired gains are 10.24 and 9.76 points, with 17 improved tasks in each run and nine versus ten regressions.

The direction of the average gain repeats for every method. SkillEvolver's gain remains near 25 points and CoEvoSkills' near ten, while SPARK shows a larger change in gain between runs. Improvements and regressions coexist within each run, making the paired task outcomes informative alongside the aggregate score. The benchmark results quantify the utility of generated and evolved skills under paired deployment. The variant experiments below examine how these gains extend to changed tasks and which artifact-level decisions support transfer.

- **报告协议**。我们明确指定支持每个主张级别所需的谱系、对比和工件，从选定的增益到有界迁移。

问题公式化

技能修订与评估

在修订轮次 r 中，一个优化器更新当前持久的技能工件：

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

其中 s_r 是当前技能，τ_r 是一个执行轨迹，而 f_r 是可能包含验证器结果、标准答案、诊断或评估器反馈的反馈。运算符 O 表示方法完整的更新路径，包括其原生合并、验证、选择或采用机制。一个被拒绝或从未被采用的生成候选不是 s_{r+1}。

代理随后使用冻结的技能执行任务 x：

$$o_r = A(x, s_r, z_r), \tag{2}$$

其中 z_r 表示模型采样、工具使用和环境状态，而 o_r 是结果输出工件或状态变化。一个验证器将输出映射为一个分数：

$$y_r = V(x, o_r). \tag{3}$$

识别问题

大多数评估观察到：

$$y_{r+1} > y_r. \tag{4}$$

这个不等式描述了两种评估；它并不能识别持久技能学到了什么。修订后的工件可能编码了保留的行为改进，选定的分数可能取决于优化器或执行变化，或者输出可能更好地匹配验证器的测量表面。基础设施无效的观察，例如从未达到其语义断言的验证器，既不是成功机制也不是失败机制，必须被修复或排除。

因此，我们将问题分为两个：

1. **效果存在和范围：** 修订后的技能在全新部署下是否保留了行为改进，以及这种效果在受控任务变化下会延伸多远?

2. **效果归因：** 哪些反馈、修订、选择、工件、执行和测量机制导致了观察到的分数变化?

以下实验依次回答这些问题。

SkillsBench上的实验

实验设置

我们在SkillsBench（Li等人，2026年）上评估了SkillEvolver（张等人，2026a）、SPARK（周等人，2026b）和CoEvoSkills（张等人，2026b），每种方法运行两次。每次运行都将使用生成的或进化的技能执行的配对任务与相同任务的技能执行配对。每种方法的两次运行都使用相同的任务集：SkillEvolver有81个任务，SPARK有84个任务，CoEvoSkills有85个任务。

我们报告了0-100分范围内的平均验证器奖励，以及技能条件与无技能条件之间的任务内平均差异。对于包含 N 个任务的运行，配对增益是

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{5}$$

我们还统计了具有正差异、负差异和零差异的任务。

两次运行的结果

所有三种方法在两次运行中都实现了正的均值配对增益（表1）。SkillEvolver在运行1中从30.20提升到55.81，在运行2中从37.32提升到62.34，分别获得了25.61和25.02个百分点增益。相应的任务计数为27次改进与6次退化，以及26次改进与3次退化。

SPARK 在运行1中将平均奖励从 17.82 提升至 19.66，在运行2中从 14.37 提升至 20.81。其配对增益从 1.84 上升到 6.45 分，改进任务数量为 9 对 13，退化数量为 8 对 5。

CoEvoSkills 在对齐的 85-任务集上从 42.21 提升至 52.44，从 23.54 提升至 33.31。其配对增益分别为 10.24 和 9.76 分，每次运行均有 17 个改进任务，退化数量为 9 对 10。

平均收益的方向对每种方法都重复出现。SkillEvolver的收益保持在25分左右，而CoEvoSkills则更接近，SPARK则显示出运行之间更大的收益变化。每次运行中，改进和退化并存，使得成对任务的成果与综合得分一起具有信息量。基准测试结果量化了在成对部署下生成的和进化的技能的效用。下方的变体实验检验了这些收益如何扩展到改变的任务，以及哪些组件级别的决策支持迁移。

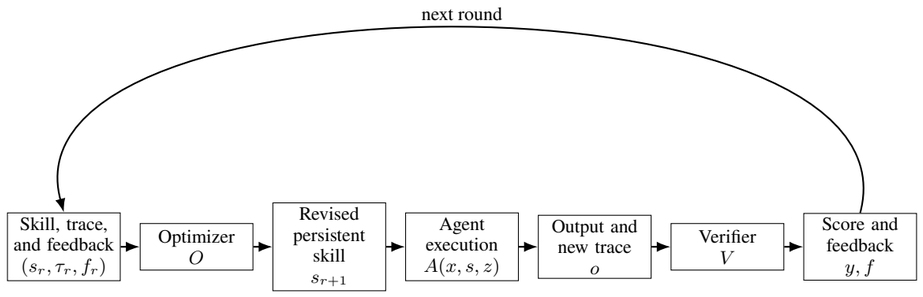


Figure 1: Minimal self-evolution lifecycle. The update operator includes any method-native validation, selection, merge, or adoption before the revised skill is retained.

Method	Run	Tasks	No skill	Skill	Δ	Improved	Regressed	Tied
SkillEvolver	1	81	30.20	55.81	+25.61	27	6	48
SkillEvolver	2	81	37.32	62.34	+25.02	26	3	52
SPARK	1	84	17.82	19.66	+1.84	9	8	67
SPARK	2	84	14.37	20.81	+6.45	13	5	66
CoEvoSkills	1	85	42.21	52.44	+10.24	17	9	59
CoEvoSkills	2	85	23.54	33.31	+9.76	17	10	58

Table 1: Two runs per method on SkillsBench. Scores are mean verifier rewards multiplied by 100; Δ is the mean paired gain in percentage points. Each method uses an identical task set across its two runs.

Generalization under Task Variations

We test how source-task skills perform when the task changes along three axes: input representation (P1), task-instance structure (P2), and coupled task requirements (P3). The reviewed collection contains 222 variants from 77 task families. Complete, unambiguous historical scores for all four conditions define a common comparison set of 198 variants from 71 families. Each variant contributes a no-skill score and scores for SkillEvolver, SPARK, and CoEvoSkills. The separate recovery evaluations are retained with their own protocol records. Table 2 summarizes paired outcomes within the historical comparison set.

SkillEvolver retains a substantial advantage on the variants: mean reward rises from 41.90 without a skill to 56.80, a paired gain of 14.90 points. It improves 46 variants and regresses on 16, compared with 29 improvements and 32 regressions for SPARK, and 25 improvements and 34 regressions for CoEvoSkills. SkillEvolver's paired gains are positive in all three variant groups: 14.89 points on P1, 17.03 on P2, and 12.85 on P3. The structural P2 group shows the largest separation: SPARK and CoEvoSkills have paired changes of -2.43 and -6.98 points, respectively.

Artifact availability differs across methods. SPARK has no generated skill for 95 of the 198 variants, whereas SkillEvolver and CoEvoSkills supply artifacts for all 198. On the 103 variants where every method supplies a skill, SkillEvolver still reaches 69.87, compared with 57.17 for SPARK and 51.46 for CoEvoSkills. Its paired advantages over these methods are 12.70 points (95% CI $[1.97, 24.35]$) and 18.42 points ($[6.68, 30.36]$). These percentile intervals use 20,000 bootstrap samples of the 36 base-task families, retaining each sampled family's variants together. The advantage therefore persists when the comparison is restricted to supplied artifacts.

Tracing the Source of Transfer

Execution traces connect the aggregate result to concrete decisions encoded in the skills. In exoplanet period detection, SkillEvolver separates candidate discovery from refinement and checks whether a dominant peak reflects stellar modulation or a transit. After the input columns are reordered in P1, the executor rejects the approximately 1.395-day modulation and recovers the accepted 5.35907-day period. When P3 requires two signals, the same frozen skill supports a narrower-duration search and refinement to 2.40052 and 5.35958 days. CoEvoSkills' end-to-end period routine encounters duration-grid errors in P1 and returns the stellar-modulation period; in P3 it selects approximately 1.40 days and its 0.70-day harmonic as separate signals. SkillEvolver succeeds on P1 and P3, while CoEvoSkills fails both. On the replacement-data P2 instance, both skills receive zero, giving the case a specific transfer pattern across the three changes.

Scheduling locates the difference at a constraint that determines correctness. SkillEvolver's artifact explicitly pre-

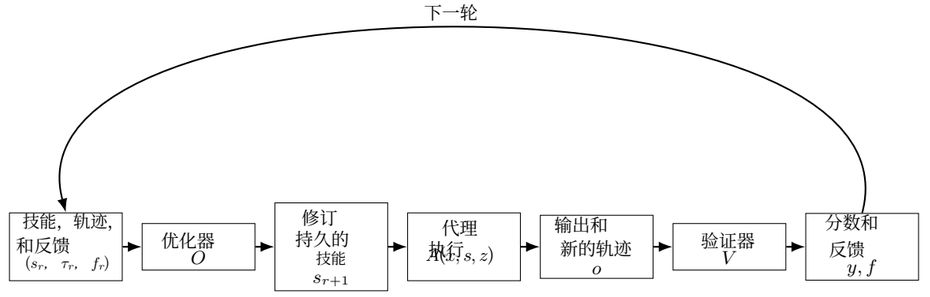


图1：最小自我进化生命周期。更新算子包括修订技能保留之前的任何方法原生验证、选择、合并或采纳。

方法	Run	任务	无技能	技能	Δ	改进	退化	Tied
SkillEvolver	1	81	30.20	55.81	+25.61	27	6	48
SkillEvolver	2	81	37.32	62.34	+25.02	26	3	52
SPARK	1	84	17.82	19.66	+1.84	9	8	67
SPARK	2	84	14.37	20.81	+6.45	13	5	66
CoEvoSkills	1	85	42.21	52.44	+10.24	17	9	59
CoEvoSkills	2	85	23.54	33.31	+9.76	17	10	58

表1：SkillsBench上每种方法运行两次。分数是验证器奖励的平均值乘以100； Δ 是百分比点上的平均配对增益。每种方法在其两次运行中使用相同的任务集。

在任务变化下的泛化能力

我们测试了当任务沿三个轴变化时，源任务技能的表现：输入表示（P1）、任务实例结构（P2）和耦合任务需求（P3）。所审查的集合包含来自77个任务系列的222个变体。所有四个条件下完整、明确的历史得分定义了一个由71个系列中的198个变体组成的共同比较集。每个变体贡献一个无技能得分以及SkillEvolver、SPARK和CoEvoSkills的得分。单独的恢复评估与其自身的协议记录一起保留。表2总结了历史比较集中成对的结果。

SkillEvolver在变体上保持显著优势：平均奖励从没有技能的41.90上升到56.80，配对增益为14.90分。它改进了46个变体，并在16个上退化，而SPARK有29个改进和32个退化，CoEvoSkills有25个改进和34个退化。SkillEvolver在所有三个变体组中的配对增益都是正的：P1为14.89分，P2为17.03分，P3为12.85分。结构P2组显示出最大的差异：SPARK和CoEvoSkills的配对变化分别为 -2.43 和 -6.98分。

不同方法下，工件可用性存在差异。SPARK 对 198 个变体中 95% 没有生成的技能，而 SkillE-volver 和 CoEvoSkills 为所有 198 个变体提供工件。在所有方法都提供技能的 103 个变体中，SkillE-volver 仍达到 69.87，相比之下 SPARK 为 57.17，CoEvoSkills 为 51.46。其相对于这些方法的配对优势分别为 12.70 分（95% CI {v1}）和 18.42 分（{v2}）。这些百分位数区间使用了 20,000 个 36 个基础任务家族的 bootstrap 样本，保留了每个采样家族的变体。因此，当比较仅限于提供的工件时，优势仍然存在。

这些优势在所有方法都提供技能的 103 个变体中，SkillE-volver 仍达到 69.87，相比之下 SPARK 为 57.17，CoEvoSkills 为 51.46。其相对于这些方法的配对优势分别为 12.70 分（95% CI $[1.97, 24.35]$）和 18.42 分（$[6.68, 30.36]$）。这些百分位数区间使用了 20,000 个 36 个基础任务家族的 bootstrap 样本，保留了每个采样家族的变体。因此，当比较仅限于提供的工件时，优势仍然存在。

追踪迁移的来源

执行轨迹将综合结果与技能中编码的具体决策联系起来。在系外行星周期检测中，SkillEvolver将候选发现与细化分开，并检查主导峰值是否反映恒星调制或凌日。在P1中输入列重新排序后，执行者拒绝大约1.395天的调制并恢复接受的5.35907天周期。当P3需要两个信号时，相同的冻结技能支持更窄持续时间的搜索和细化到2.40052和5.35958天。CoEvoSkills的端到端周期例行程序在P1中遇到持续时间网格错误，并返回恒星调制周期；在P3中它选择大约1.40天及其0.70天的谐波作为单独信号。SkillEvolver在P1和P3中成功，而CoEvoSkills两个都失败。在替换数据P2实例中，两个技能都收到零，使该案例在三次变化中呈现出特定的迁移模式。

调度定位了约束差异，该约束决定了正确性。SkillEvolver 的工件明确预先

Method	Variants	No skill	Skill	Δ	Improved	Regressed	Tied
A. Common historical comparison set: 71 task families							
SkillEvolver	198	41.90	56.80	+14.90	46	16	136
SPARK	198	41.90	43.63	+1.73	29	32	137
CoEvoSkills	198	41.90	39.34	−2.56	25	34	139
B. All three methods supply a skill: 36 task families							
SkillEvolver	103	51.21	69.87	+18.67	25	6	72
SPARK	103	51.21	57.17	+5.96	18	14	71
CoEvoSkills	103	51.21	51.46	+0.25	16	15	72

Table 2: Descriptive comparison on matched task variants. Scores and paired gains use the same scale as Table 1. Panel B restricts Panel A to variants whose launch records confirm an available skill for every method, separating artifact availability from performance when artifacts are supplied.

serves baseline-relative right shifts and precedence-aware local minimality. The executor carries these rules across renamed inputs, a replacement scheduling instance, and an added machine-handoff requirement, passing all three variants; SPARK also passes all three. CoEvoSkills passes P3, but its P1 output fails the local-minimality check: an operation starts at 29 with an anchor of 24 and can move earlier. The other structural and policy checks pass. This trace-based attribution connects the outcome to preservation of a repair invariant. Together, the cases illustrate a transferable skill as a set of diagnostic and constraint-preserving decisions that the executor adapts to the new task.

Identifying Retained Skill Effects

Evaluation Design and Contrasts

We analyze three persistent-skill systems with distinct revision mechanisms. SkillRevise converts execution traces and diagnoses into bounded repair principles (Liu et al. 2026b). SkillHone accumulates practice history and iteratively reflects on the persistent skill (Li and Hu 2026). SkillOpt produces structured bounded edits and uses a validation gate and rejected buffer before adoption (Yang et al. 2026). The systems are studied as representative mechanisms, not as directly comparable leaderboard entries.

At the paper level, the evidence is exploratory: individual interventions use frozen inputs and predeclared gates, but the combined studies are designed to identify candidate mechanisms rather than estimate benchmark-wide prevalence. A targeted 24-task SkillsBench panel tests whether historical outcomes survive fresh deployment. The panel is stratified to expose stability failures and method disagreements; it is not a random sample. Three controlled mechanism cases then compare complete parent and final artifacts under predeclared variations. Within each contrast, we hold the executor model, harness, task instance, budget, and verifier version fixed unless one is the named intervention; exact prompts, hashes, seeds, and stopping decisions are preserved in the artifact package. The task is the primary analysis unit, while repeated runs are deployments of the same task-condition pair.

Four comparisons must remain distinct. Historical-pass to fresh-repeat comparisons test retention of a selected outcome, not advantage over a baseline. Final-to-no-skill comparisons estimate fresh utility, not the effect of revision. Only actual-parent to final comparisons identify a revision effect. Applying that comparison to controlled variants tests bounded transfer, not population-average benefit.

方法	变体	无技能	技能	Δ	改进	退化	Tied
A. 常见历史比较集：71 任务族							
SkillEvolver	198	41.90	56.80	+14.90	46	16	136
SPARK	198	41.90	43.63	+1.73	29	32	137
CoEvoSkills	198	41.90	39.34	−2.56	25	34	139
B. 三种方法均提供技能：36 任务族							
SkillEvolver	103	51.21	69.87	+18.67	25	6	72
SPARK	103	51.21	57.17	+5.96	18	14	71
CoEvoSkills	103	51.21	51.46	+0.25	16	15	72

表 2：匹配任务变体的描述性比较。分数和配对增益使用与表 1 相同的刻度。面板 B 将面板 A 限制为启动记录确认每种方法都有可用技能的变体，在提供工件时将工件可用性与性能分离。

提供基线相对的右移位和考虑优先级的局部最小性。执行器将这些规则跨重命名输入、一个替换调度实例以及一个添加的机器转交需求传递，传递了所有三种变体；SPARK 也传递了所有三种。CoEvoSkills 传递 P3，但其 P1 输出未通过局部最小性检查：一个操作从 29 开始，锚点为 24，可以移到更早的位置。其他结构和策略检查均通过。这种基于轨迹的归因将结果与修复不变量的保持联系起来。这些案例共同说明了可迁移技能是一组诊断和保持约束的决策，执行器将其适应到新任务中。

识别保留的技能效应

评估设计与对比

我们分析了三种具有不同修订机制的持久技能系统。SkillRevise 将执行轨迹和诊断转换为有界的修复原则（Liu 等人，2026b）。SkillHone 累积实践历史并迭代地反思持久技能（Li 和 Hu，2026）。SkillOpt 生成结构化的有界编辑，并在采用前使用验证门和拒绝缓冲区（Yang 等人，2026）。这些系统作为代表性机制进行研究，而不是作为可直接比较的排行榜条目。

在论文层面，证据是探索性的：单个干预使用冻结的输入和预声明的门，但综合研究旨在识别候选机制，而不是估计基准范围内的普遍性。一个有针对性的24任务SkillsBench面板测试历史结果是否能在新的部署中存活。该面板按层次划分，以暴露稳定性失败和方法分歧；它不是一个随机样本。然后比较三个受控机制案例，在预声明的变化下比较完整的父级和最终工件。在每个对比中，我们保持执行者模型、测试框架、任务实例、预算和验证器版本固定，除非它是命名的干预；精确的提示、哈希值、种子和停止决策都保存在工件包中。任务是主要分析单元，而重复运行是相同任务条件对的部署。

四个比较必须保持区分。历史到新鲜重复的比较测试选定结果的保留，而不是相对于基线的优势。最终到无技能的比较比较估计的是新效用，而不是修订的效果。只有实际父代到最终比较才能识别修订效果。将这种比较应用于受控变体测试的是有界迁移，而不是群体平均收益。

Panel and contrast	Main result	Supported inference and boundary
Panel A: Retention under fresh deployment		
Historical pass → fresh repeats	12/24 tasks flip within at least one frozen condition; selected-outcome retention is 47.1% for SkillRevise and 55.0% for SkillHone	A selected pass is not a deployment probability; denominators differ and are not a method ranking.
SkillRevise final → no-skill	16/48 vs. 14/48; +4.2 pp, 95% CI $[-10.4, 20.8]$	No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate.
SkillHone final → no-skill	12/48 vs. 14/48; −4.2 pp, 95% CI $[-20.8, 14.6]$	No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate.
Panel B: Revision effects under controlled variations		
Invoice parent → final	$0/12 \rightarrow 12/12$; three new same-contract corpora: $0/9 \rightarrow 9/9$	Stable but narrow same-contract transfer.
Energy parent → final	Original task plus 5% parameter variant: $1/6 \rightarrow 5/6$	Directional parameter transfer in a small stochastic case.
SkillOpt SearchQA parent → final	Independent held-out $9/24 \rightarrow 17/24$; stable-transfer gate fails	Local held-out gain, not stable general procedure learning.

Table 3: Evidence for retained skill effects. Panel A contains 24 targeted tasks, three conditions, and two fresh deployments per condition (144/144 behavior-valid runs). Different historical-retention denominators preclude horizontal method ranking. The no-skill condition calibrates execution and selection variability; final-versus-no-skill does not identify revision effects.

Retention under Fresh Deployment

This experiment tests whether historically selected outcomes survive when the final artifact is frozen and redeployed. Holding task, skill, and condition fixed does not make execution deterministic. Across the 24-task panel, 12 tasks change success status between their two repeats in at least one condition. Aggregate fresh success is 14/48 without a skill, 16/48 with the SkillRevise final artifact, and 12/48 with the SkillHone final artifact. Both matched-lift intervals cross zero (Table 3).

These results establish an identification failure in the common reporting practice: cached or selected success cannot be read as stable deployment capability. They do not establish that either revision method is ineffective. The panel is purposive, and it lacks each method's actual parent condition. Its proper conclusion is narrower: on this sample, neither final-versus-no-skill comparison establishes a clear average fresh-deployment advantage, and fresh repeats are necessary before a selected score is called retained.

Revision Effects under Controlled Variations

These mechanism probes test whether the actual parent-to-final revision effect persists under predeclared task changes. In Invoice, the complete final skill repairs all 12 matched deployments and retains the effect on nine deployments drawn from three disjoint corpora that preserve the contract while changing the documents. This is strong evidence of a stable, same-contract revision effect, but not of broad cross-contract generalization.

The Energy case provides weaker but directionally consistent evidence: the complete final artifact improves the original setting and a 5% parameter variant in five of six deployments, compared with one of six for the parent. The sample is too small to support a prevalence or generality claim. In SkillOpt SearchQA, the final artifact improves independent held-out accuracy from 9/24 to 17/24, yet fails the predeclared stability threshold. It therefore supports local held-out benefit rather than stable transferable procedure learning.

Taken together, revised skill artifacts can produce retained behavioral effects, but a selected benchmark pass alone does not identify such an effect, and the effects established so far remain narrow rather than broadly generalizable.

Attributing Observed Skill Gains

Lifecycle Interventions

The experiments in this section intervene at different lifecycle stages. They do not estimate one common effect size. Each intervention changes one object while holding its relevant lineage fixed: feedback-view studies hold the parent and failures fixed; optimizer-repeat studies hold the input evidence fixed; gate audits preserve accepted and rejected candidates; artifact surgery holds the surrounding artifact fixed; and verifier counterfactuals hold the output fixed.

面板与对比	主要结果	支持的推理与边界
面板A：新部署下的留存率		
历史通过 → 新重复	12/24任务至少翻转一个冻结条件；选择结果保留 47.1% for 技能修订 and 55.0% for 技能精炼	一个选定的通过不是部署概率；分母不同，并且不是一种方法排名。
技能修订最终 → 无技能	16/48 vs. 14/48； +4.2pp，95% CI $[-10.4, 20.8]$	在此目标面板上没有明显的全新效用优势；不是从父级到最终修订的估计。
技能精炼最终 →无技能	12/48 与 14/48； −4.2 pp，95% CI $[-20.8, 14.6]$	在该目标面板上，没有明显的全新效用优势；不是从父版到最终修订的估计。
面板 B：受控变化下的修订效果		
发票父版 →最终	$0/12 \rightarrow 12/12$; 三个新的相同合同语料库: $0/9 \rightarrow 9/9$	稳定但狭窄的相同合同迁移。
能量父 →最终	原始任务加5%参数变体: $1/6 \rightarrow 5/6$	小随机情况下的方向参数迁移。
SkillOpt SearchQA 父 → 最终	独立保留集 $9/24 \rightarrow 17/24$; 稳定迁移门控失败	本地保留集增益，非稳定泛化程序学习。

表 3：保留技能效应的证据。面板 A 包含 24 个目标任务、三个条件和每个条件两个新部署（144/144 行为有效运行）。不同的历史保留分母排除了横向方法排名。无技能条件校准执行和选择变异性；最终与无技能的对比无法识别修订效应。

新部署下的保留

这项实验测试了当最终产物被冻结并重新部署时，历史上选择的成果是否仍然存在。保持任务、技能和条件固定并不能使执行变得确定。在24项任务面板中，12项任务在至少一个条件下两次重复时改变了成功状态。没有技能时，总新成功率为14/48；使用SkillRevise最终产物时为16/48；使用SkillHone最终产物时为12/48。两个匹配提升间隔都穿过零（表3）。

这些结果确立了一种在常见报告实践中存在的识别失败：缓存的或选择的成功不能被解读为稳定的部署能力。它们并未确立任何修订方法都无效。该面板是故意的，并且缺少每种方法的实际父条件。其恰当的结论更狭窄：在这个样本中，最终与无技能的对比并未确立清晰的平均新部署优势，在称为保留之前，新重复是必要的。

受控变化下的修订效应

这些机制探测测试实际父条件到最终修订效应是否在预声明的任务变化下持续存在。在发票中，完整的最终技能修复了所有 12 个匹配部署，并保留了从三个保持合同但更改文档的独立语料库中抽取的九个部署的效果。这是稳定、同合同修订效应的强有力证据，但不是广泛跨合同泛化的证据。

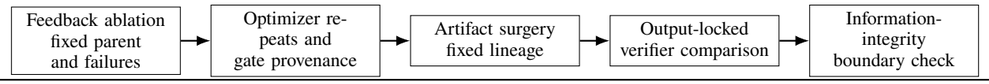
能源案例提供了较弱但方向上一致的证据：完整的最终成果改进了原始设置，在六个部署中的五个里使用了5%参数变体，而父案例则只有一个部署使用了。样本太小，无法支持普遍性或一般性主张。在SkillOpt SearchQA中，最终成果将独立保留的准确率从9/24提高到17/24，但未能通过预定义的稳定性阈值。因此它支持局部保留收益，而非稳定的可迁移程序学习

综合来看，修订后的技能工件可以产生持续的行为效应，但单独的选定基准测试并不能识别这种效应，而且迄今为止建立起来的效应仍然比较狭窄，而非广泛普适。

归因于观察到的技能提升

生命周期干预

本节中的实验在不同的生命周期阶段进行干预。它们不估计一个共同的效果量。每次干预都会改变一个对象，同时保持其相关谱系固定：反馈-视图研究保持父对象和失败固定；优化器-重复研究保持输入证据固定；门控审计保留接受和拒绝的候选对象；工件手术保持周围工件固定；而验证器反事实保持输出固定。



Lifecycle stage	SkillRevise	SkillHone	SkillOpt
Feedback and revision	Diagnosis ablation; revision traces	Feedback-view ablation	Feedback branches; optimizer repeats
Selection and adoption	Revision history	Iterative honing history	Native validation gate; accepted and rejected candidates
Artifact mediation	Not yet matched at the same strength	Invoice and Energy surgery	SearchQA component surgery
Measurement	No output-locked comparison yet	Radar verifier swap	Spreadsheet dual scoring
Information integrity	Artifact forensic	Artifact forensic	Wrong-metadata and oracle controls

Figure 2: Lifecycle attribution design and current evidence coverage. The upper panel locates each intervention; the lower panel records method coverage. Populated cells do not imply equivalent strength, and missing cells bound cross-method claims.

Figure 2 makes the current coverage and its gaps explicit. The shared audit logic is that feedback access, candidate revision, validation acceptance, adoption into the persistent artifact, and retained behavioral improvement are distinct events.

Revision and Selection Pathways

These interventions test whether the observed endpoint depends on the feedback view, optimizer sample, or selection gate. In SkillRevise, the complete three-revision pipeline succeeds on 53/86 tasks, compared with 28/86 when diagnosis is removed. Task-level discordance is 26 gains versus one loss, and 2,326 of 3,648 revision entries explicitly use verifier evidence. This demonstrates that diagnosis changes the selected endpoint and that the revision process is strongly evaluator-facing. It does not, by itself, establish which changes are retained under fresh deployment.

SkillHone separates access to feedback from stable benefit. Detailed sanitized, binary, and redacted-tail feedback obtain first-deployment success on 4/12, 4/12, and 5/12 cases, while four of 18 repeated cells flip. Feedback can trigger revisions, but these data do not support a stable advantage for one feedback view.

SkillOpt further separates candidate production from attribution and adoption. In matched SearchQA optimizer repeats, raw-gold and outcome-only views each produce four applied edits. Yet the preregistered between-condition effect size is only $D = .044 < .20$, despite weak lexical decodability, so a single patch cannot be cleanly attributed to feedback view. Across outcome-only and trajectory-only branches, all 6/6 candidates change and 3/6 pass the gate, but every acceptance is driven by the same Blackwater validation item. In OfficeQA, a selected one-item gain disappears in fresh matched deployment. Thus edit production, gate acceptance, and retained improvement cannot be collapsed into one event.

Artifact-Level Mediation

This experiment tests whether a feedback-derived component, rather than unrelated edits in the full artifact, causes the behavior after adoption. Let P denote the complete parent, F the complete final artifact, $P+B$ the parent with component B, and $F-B$ the final artifact with that component removed.

Case	Result	Maximum claim
Invoice	$P = 0/12$, $P+B = 12/12$, $F-B = 0/12$, $F = 12/12$	B is necessary and sufficient for the narrow same-contract effect.
Energy	Local mediation in the original task; necessity and sufficiency do not survive the variant	Local component effect, not transferable mediation.
SearchQA	9 repairs, 0 regressions; stable success 7/12, below the strong gate	Directional mediation on part of the held-out set.

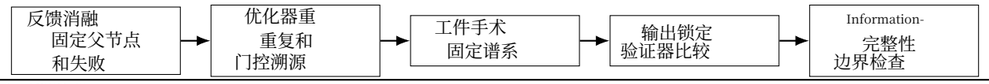
Table 4: Artifact-level mediation evidence.

These interventions establish that an adopted artifact component can cause behavior. They do not show that the same mechanism is prevalent across methods. In particular, Skill-Revise lacks a matched surgery of equal strength, and the current cases should be read as existence and boundary evidence.

Measurement Dependence under Fixed Outputs

This counterfactual tests whether a verdict changes when the output is fixed and only the measurement changes. In the Radar family, all 12 reconstructed outputs reproduce their historical original-verifier outcomes. Of six original passes, only one remains a pass under a reference-calibrated strict verifier; the five valid flips arise from specification-implied checks. The same behavior can therefore receive different valid verdicts because the measurement surfaces differ.

Measurement dependence can also operate in the other direction. For identical Spreadsheet workbook outputs, the reported value-based evaluator scores the complete SkillOpt



生命周期阶段	技能修订	技能精炼	SkillOpt
反馈与修订	诊断消融；修订轨迹	反馈视图消融	反馈分支；优化器
选择与采纳	修订历史	迭代打磨历史	重复 本地验证关卡；已接受 和拒绝的候选人
工件调解	尚未在相同 力量	发票和能源手术	SearchQA组件手术
测量	尚未有输出锁定比较	雷达验证器交换	电子表格双重评分
信息完整性	文物取证	文物取证	错误元数据和预言机 控制

图2：生命周期归因设计及当前证据覆盖范围。上层面板定位每个干预措施；下层面板记录方法覆盖范围。填充的单元格不表示同等强度，缺失的单元格界定跨方法主张。

图2使当前的覆盖范围及其空白显式化。共享的审计逻辑是：反馈访问、候选修订、验证接受、持久工件采纳以及保留的行为改进是不同的独立事件。

修订与选择路径

这些干预措施测试观察到的终点是否取决于反馈视图、优化器样本或选择门。在SkillRevise中，完整的三重修订管道在53/86个任务上成功，而移除诊断时则为28/86。任务级不一致性为26个收益与1个损失，且3,648个修订条目中有2,326个明确使用验证器证据。这表明诊断改变了选定的终点，且修订过程强烈面向评估器。它本身并不能确定哪些变更在全新部署下被保留。

SkillHone将反馈访问与稳定收益分离。详细的清理、二进制和截断尾部反馈在4/12、4/12和5/12个案例中获得首次部署成功，而18个重复单元中有4个发生翻转。反馈可以触发修订，但这些数据不支持某一反馈视图的稳定优势。

SkillOpt 进一步将候选生成与归因和采纳分开。在匹配的 SearchQA 优化器重复中，原始金标和仅结果视图各产生四个应用编辑。然而，预先注册的跨条件效应量仅为 $D = .044 < .20$，尽管词汇解码能力较弱，因此单个补丁无法清晰地归因于反馈视图。在仅结果和仅轨迹分支中，所有 6/6 候选都发生变化，且 3/6 通过了门控，但每次接受都是由同一个 Blackwater 验证项驱动。在 OfficeQA 中，一个选定的单项增益在新的匹配部署中消失。因此，编辑生成、门控接受和保留的改进不能合并为一个事件。

Artifact-Level Mediation

该实验测试的是，是否是源自反馈的组件而非完整工件中的无关修改导致了采纳后的行为。令 P 表示完整的父项， F 完整的最终工件、 $P+B$ 带组件的父项B，以及 $F-B$ 移除了该组件的最终工件。

Case	结果	最大主张
发票	$P = 0/12$, $P+B = 12/12$, $F-B = 0/12$, $F = 12/12$	B 是必要且充分的。
能量	本地调解 在原始版本中 任务；必要性 和充分性 不会存活 变异版本	狭窄的相同合同效应。 局部组件效应，不 可迁移的中介。
SearchQA	9 修复，0 退化； 稳定成功 7/12，低于 强门	部分方向中介 在保留集上。

表4：指标级中介证据。

这些干预表明，一个采纳的工件组件可以导致行为。它们并未表明相同的机制在所有方法中都普遍存在。特别是，Skill-Revise 缺乏同等强度的匹配手术，当前案例应被视为存在性和边界证据。

固定输出下的测量依赖

这个反事实测试了当输出固定且仅测量变化时，裁决是否会改变。在Radar系列中，所有12个重建输出都重现了它们的历史原始验证器结果。在六个原始通过中，只有一个在参考校准严格验证器下仍然通过；五个有效翻转来自规范暗示的检查。因此，相同的行为可能因为测量表面不同而获得不同的有效裁决。

测量依赖也可以反向运作。对于相同的电子表格工作簿输出，报告的基于值的评估器评分了完整的SkillOpt

artifact at 10/12, whereas joint value-and-formula recalculation scores it at 8/12. Representation, cache materialization, or formula inspection can therefore change what the evaluator observes. This dual scoring is not identical in design to the Radar output-locked swap, but it reinforces the narrower point that score is a property of output plus measurement, not behavior alone.

Neither result proves semantic failure, exploitation, or benchmark-wide prevalence. To identify evolution-induced verifier dependence, one would additionally need matched no-skill, parent, and final outputs under both original and calibrated verifiers, and test whether the parent-to-final verifier gap increases.

Information-Integrity Boundary

This supporting check tests whether restricted answer or oracle information enters and controls the artifact. High-confidence direct-answer or oracle matches are absent from 86 SkillRevise and 85 SkillHone natural artifacts. SkillOpt conditions containing the feedback-derived answer-surface section follow incorrect metadata in 0/15 probes. In contrast, an unsafe hidden-oracle positive control changes probe success from 0/5 to 5/5 and leaves private-oracle or gold markers.

These checks show that the audit can detect direct exposure when it is deliberately introduced, while the natural artifacts do not support direct copying as a universal explanation. They do not prove the absence of implicit answer-following policies or all forms of leakage.

Observed score gains can arise at multiple lifecycle stages: feedback shapes candidate revisions, selection can amplify narrow improvements, adopted artifact components can mediate real behavior, and valid verifiers can disagree on fixed outputs. These mechanisms must be identified separately before a score gain is interpreted as skill learning.

From Reported Gains to Auditable Claims

Evidence Requirements and Claim Boundaries

The audit is not a scalar ladder from *safe* to *unsafe*. Its steps answer independent questions and impose distinct claim boundaries.

Audit step	Required test → maximum supported claim
Validity	Execution and verifier status plus assertion coverage → eligible observation
Retention	Frozen actual-parent/final fresh repeats → retained artifact effect
Behavioral scope	Entity, parameter, input, contract, or family variants → bounded controlled transfer
Evolution pathway	Feedback ablation, optimizer repeats, and gate provenance → pathway dependence or selection mediation
Measurement	Output-locked calibrated verifier comparison → measurement dependence
Information integrity	Provenance, wrong-metadata, and positive control → exploitation or confirmed leakage, when supported

Table 5: Evidence requirements and claim boundaries. The rows answer complementary questions rather than forming a scalar risk ladder.

A selected gain is weaker than a retained artifact effect; a retained effect on one instance is weaker than controlled transfer. Evolution-pathway and measurement claims remain meaningful even when retention is not established. Conversely, verifier-aware language or evaluator dependence must not be relabeled as leakage without restricted-information provenance and behavioral evidence.

Lifecycle Reporting Protocol

An auditable self-evolution report should preserve the following minimum evidence:

- **Before optimization:** parent artifact, task and split identity, failure pool, environment, and verifier version.
- **During revision:** optimizer-visible traces and feedback, candidate patches, optimizer samples, prompts, and usage.
- **At selection and adoption:** accepted and rejected artifacts, gate items, per-item decisions, and exact adopted artifact.
- **At deployment:** frozen artifact, actual-parent and no-skill conditions, fresh repeats, controlled variants, and output bytes.
- **At scoring:** verifier logs, assertion coverage, reference calibration, and any alternative verifier applied to the same output.
- **For cost:** separate optimizer, executor, verifier, and invalid-attempt ledgers.

The complete artifact appendix should additionally record hashes, commands, seeds, environment revisions, timeouts, and stopping decisions. This protocol turns a post-evolution score into an auditable lineage and limits the paper's language to the strongest matched contrast actually performed.

Related Work

Skill Evolution and Empirical Evaluation

Agent-skill research spans generation, revision, selection, retrieval, organization, and reinforcement learning. SkillsBench and SWE-Skills-Bench compare skill and no-skill utility across diverse or software-engineering tasks (Li et al. 2026; Han et al. 2026). SkillLearnBench evaluates continual skill generation (Zhong et al. 2026), while SkillRevise, SkillHone, SkillOpt, SkillRL, and CoEvoSkills propose different revision or learning loops (Liu et al. 2026b; Li and Hu 2026; Yang et al. 2026; Xia et al. 2026; Zhang et al. 2026b). Recent empirical work broadens the object of study: From Raw Experience to Skill Consumption separates experience generation, extraction, and consumption (Huang et al. 2026); How Well Do Agentic Skills Work in the Wild studies realistic skill retrieval (Liu et al. 2026a); SkillCoach evaluates process-level skill use (Zhu et al. 2026); and SkillJuror isolates the effect of skill organization (Chen et al. 2026).

These studies show that outcome-only evaluation is already recognized as incomplete. Our focus is narrower and complementary: when a persistent skill is revised and the

工件在10/12处被评估，而联合值和公式重新计算则将其评估为8/12。因此，表示、缓存实现或公式检查可能会改变评估器所观察到的内容。这种双重评分在设计上与Radar输出锁定交换并不相同，但它加强了更狭窄的观点：分数是输出与测量的属性，而不仅仅是行为的属性。

这两个结果都无法证明语义错误、利用行为或基准范围内的普遍存在。要识别由进化引起的验证器依赖性，还需要在原始和校准的验证器下获得匹配的无技能、父代和最终输出，并测试父代到最终的验证器差距是否增加。

信息完整性边界

这项辅助检查测试受限答案或预言机信息是否进入并控制了工件。86个SkillRevise和85个SkillHone自然工件中均未出现高置信度的直接答案或预言机匹配。包含反馈衍生答案表面部分的SkillOpt条件在15个探测中0/15次遵循了错误的元数据。相比之下，一个不安全的隐藏预言机正向控制将探测成功率从0/5变为5/5，并保留了私有预言机或黄金标记。

这些检查表明，审计可以在故意引入时检测直接暴露，而自然工件并不支持直接复制作为普遍解释。它们无法证明隐式答案跟随策略或所有形式的泄漏不存在。

观察到的分数提升可能出现在多个生命周期阶段：反馈会塑造候选修订版本，选择可以放大狭窄的改进，采用的艺术品组件可以调解实际行为，而有效的验证器可能对固定输出存在分歧。在将分数提升解释为技能学习之前，必须单独识别这些机制。

从报告的收益到可审计的声明证据要求和声明边界

审计并非从安全到不安全的标量阶梯。它的步骤回答独立问题，并施加不同的主张边界。

审计步骤	所需测试 → 最大支持的声明
有效性	执行和验证器状态加上断言覆盖率 →合格
保持	观察 冻结实际父/最终新鲜重复 →保留的工件
行为	效果 实体、参数、输入、合约或家族变体 → 受限
范围	受控转移
进化	反馈消融、优化器重复和门控溯源 →
路径	路径依赖或选择介导
测量	输出锁定校准验证器比较 → 测量
信息	依赖 溯源、错误元数据和正向控制 →
完整性	利用或已确认的泄露，在受支持时

表 5：证据要求和声明边界。行回答补充性问题，而不是形成一个标量风险阶梯。

一个选定的收益比保留的文物效应弱；对一个实例的保留效应比受控迁移弱。即使保留未建立，进化路径和测量主张仍然有意义。相反，验证器感知语言或评估器依赖不得在没有受限信息来源和行为证据的情况下被重新标记为泄漏。

生命周期报告协议

可审计的自进化报告应保留以下最低证据：

- **优化前：** 父工件、任务和分割身份、失败池、环境和验证器版本。
- **修订期间：** 优化器可见的轨迹和反馈、候选补丁、优化器样本、提示和使用。
- **选择与采纳：** 接受的和拒绝的工件、门控项、单项决策以及精确采纳的工件。
- **在部署时：** 冻结的工件、实际父和技能无条件、新鲜重复、受控变体以及输出字节。
- **在评分时：** 验证器日志、断言覆盖率、参考校准以及应用于相同输出的任何替代验证器。
- **关于成本：** 单独的优化器、执行器、验证器以及无效尝试账本。

完整的工件附录还应记录哈希值、命令、种子、环境修订、超时和停止决策。该协议将进化后的评分转化为可审计的谱系，并限制了论文的语言仅限于实际执行的最强匹配对比。

相关工作

技能进化和实证评估

代理-技能研究涵盖生成、修订、选择、检索、组织和强化学习。技能-基准和SWE-技能-基准比较技能和无技能在不同或软件工程任务中的效用（Li等人，2026；Han等人，2026）。SkillLearnBench评估持续技能生成（Zhong等人，2026），而SkillRevise、SkillHone、SkillOpt、SkillRL和CoEvoSkills提出不同的修订或学习循环（Liu等人，2026b；Li和Hu，2026；Yang等人，2026；Xia等人，2026；Zhang等人，2026b）。最近的实证工作扩展了研究对象：从原始经验到技能消费将经验生成、提取和消费分开（Huang等人，2026）；《代理技能在现实中的表现如何》研究现实技能检索（Liu等人，2026a）；SkillCoach评估过程级技能使用（Zhu等人，2026）；SkillJuror隔离技能组织的影响（Chen等人，2026）。

这些研究表明，仅基于结果的评估已被公认为是不完整的。我们的关注点更狭窄且具有补充性：当一项持久的技能被修订时，

post-revision score rises, what evidence identifies a retained artifact effect rather than a selected or measurement-dependent endpoint? We therefore organize evidence by lifecycle contrast rather than by method score.

Evaluation and Attribution of Agent Improvement

Agent evaluation increasingly examines process quality, sequential adaptation, verifier reliability, and counterfactual behavior. Counterfactual Trace Auditing measures how a provided skill changes an agent trajectory (Zhou et al. 2026a). SEA-Eval evaluates self-evolving agents beyond isolated episodes (Jiang et al. 2026). Co-evolving verifier work and grader audits study interactions between the policy and its measurement signal (Zhang et al. 2026b,c). More broadly, reward hacking and test-set contamination research demonstrates that performance on a proxy or exposed test need not imply the intended capability (Amodei et al. 2016; Oren et al. 2023).

We combine these concerns within the lifecycle of persistent skill revision. The central distinction is between retention, behavioral scope, evolution-and-selection pathway, and output-level measurement dependence. Information-integrity checks play a supporting role: they bound direct answer or oracle exposure, but do not convert every verifier-aware adaptation into a leakage claim.

Limitations

Our evidence is mechanism-focused rather than a benchmark-wide prevalence study. The 24-task retention panel is purposive and contains only two fresh deployments per condition; it exposes instability but cannot estimate deployment rates. The controlled Invoice, Energy, SearchQA, Radar, Spreadsheet, and OfficeQA probes use small, heterogeneous task families rather than a fully crossed method-by-intervention design. SkillRevise, for example, lacks matched artifact surgery and output-locked measurement evidence of the same strength, while the strongest verifier swap is concentrated in the Radar family. We therefore do not claim that any observed mechanism is universal or that one method outperforms another.

Several conclusions are also bounded by observability. Output-locked comparisons identify measurement dependence but do not determine which verifier best captures intended behavior without an external specification. Artifact forensics and wrong-metadata controls can rule out simple direct exposure, but not implicit answer-following policies or every form of contamination. Finally, our scope is frozen-agent, feedback-driven systems with persistent external artifacts; parameter-updating self-evolution, multimodal skills, and online environments may exhibit different lifecycle dynamics. Confirmatory evidence will require a shared sampling plan that fully crosses actual parent and final artifacts, fresh repeats, controlled variants, and calibrated verifiers.

Conclusion

We study whether benchmark gains from self-evolving persistent skills correspond to retained behavioral improvement and, independently, which lifecycle mechanisms produce those gains. The variant comparison shows a substantial transfer advantage for SkillEvolver, including when all methods supply artifacts. Execution traces connect successful cases to reusable diagnostic steps and preservation of task constraints. Our audits show that revised skill artifacts can produce real behavioral changes, but selected gains can disappear under fresh deployment and the transfer established so far remains narrow. Feedback, optimizer variation, validation selection, artifact content, execution variability, and verifier choice can each shape the observed endpoint.

A higher benchmark score should therefore be reported only at the strongest level supported by matched evidence: a selected gain, a retained artifact effect, bounded controlled transfer, evolution-pathway dependence, or measurement dependence. We provide a lifecycle audit that links each claim to the required intervention and reporting evidence. Claims of transferable skill learning require frozen parent-to-final improvement that survives fresh deployment and predeclared task variation under valid measurement; weaker evidence should be reported more narrowly.

References

Amodei, D.; Olah, C.; Steinhardt, J.; Christiano, P.; Schulman, J.; and Mané, D. 2016. Concrete Problems in AI Safety. arXiv:1606.06565.

Chen, Z.; Guo, Z.; Huang, B.; Lu, B.; Lin, J.; Zhou, Y.; and Zhang, W. 2026. SkillJuror: Measuring How Agent Skill Organization Changes Runtime Behavior. arXiv:2606.11543.

Han, T.; Zhang, Y.; Song, W.; Fang, C.; Chen, Z.; Sun, Y.; and Hu, L. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? arXiv:2603.15401.

Huang, Z.; Xu, J.; Yang, Y.; Gong, Z.; Yang, Q.; Tian, M.; Wang, X.; Lv, C.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, X.; Chen, D.; Zheng, X.; and Luo, C. 2026. From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills. arXiv:2605.23899.

Jiang, S.; Ma, L.; Hong, Z.; Wang, K.; Lu, Z.; Wang, T.; Chen, S.; Zhang, J.; Pan, T.; Li, W.; Liang, J.; and Xiao, Y. 2026. SEA-Eval: A Benchmark for Evaluating Self-Evolving Agents Beyond Episodic Assessment. arXiv:2604.08988.

Li, X.; Liu, Y.; Chen, W.; You, B.; Di, Z.; He, Y.; Zheng, S.; Choe, K. W.; Sun, J.; Wang, S.; Tao, C.; Li, B.; Zhao, X.; Geng, H.; Wu, X.; Zhou, J.; Chen, X.; Xing, H.; Li, Y.; Zeng, Q.; Wang, D.; Wang, Y.; Chaim, R. B.; Jiang, P.; Shen, H.; Kong, L.; Liu, X.; Wang, R.; Liu, X.; Li, J.; Lan, X.; Lin, Y.; Ye, W.; He, J.; Li, S.; Zhang, Y.; Gao, Y.; Li, Y.; Ma, Z.; Jing, L.; Wang, T.; Li, K.; Xue, Y.; Lyu, H.; He, Y.; Tian, Y.; Wu, S.; Wang, B.; Gao, Y.; Chen, B.; Liu, L.; Cheng, S.; Bao, J.; Tong, S.; Xu, S.; Zhuo, T. Y.; Ye, T.; Qi, Q.; Li, M.; Liao, L.; Tan, Z.; Shi, C.; Tang, X.; Tankasala, S.; Yuan, B.; Qian, Y.; Tu, J.; Wang, C.; Sun, Y.; Wang, W.; Taylor, A.; Yang, Z.; Guan, C.; Dong, Z.; Zhang, X.; Dillmann, S.; chung Lee, H.; and Song, D. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. arXiv:2602.12670.

Li, Z.; and Hu, Y. 2026. SkillHone: A Harness for Continual Agent Skill Evolution Through Persistent Decision History. arXiv:2606.08671.

修订后的得分上升，哪些证据能识别出保留的工件效应，而不是被选择或测量依赖的终点？因此，我们按生命周期对比来组织证据，而不是按方法得分来组织。

代理改进的评估与归因

代理评估越来越多地考察过程质量、序列适应、验证器可靠性和反事实行为。反事实轨迹审计测量提供的技能如何改变代理轨迹（Zhou 等人 2026a）。SEA-Eval 超越孤立场景评估自进化代理（Jiang 等人 2026）。协同进化的验证器工作和评分器审计研究策略与其测量信号之间的交互（Zhang 等人 2026b,c）。更广泛地说，奖励攻击和测试集污染研究证明，在代理或暴露的测试上的表现不一定意味着预期的能力（Amodei 等人 2016；Oren 等人 2023 ）。

我们将这些关注点结合在持久技能修订的生命周期中。核心区别在于保持、行为范围、进化与选择路径以及输出级测量依赖。信息完整性检查起辅助作用：它们限制直接答案或预言机暴露，但不会将每个验证器感知的适应都转化为泄露声明。

局限性

我们的证据是机制导向的，而非跨基准的普遍性研究。24项任务保持性面板是经过精心设计的，每个条件下仅包含两次新鲜部署；它暴露了不稳定性，但无法估计部署率。受控的Invoice、Energy、SearchQA、Radar、Spreadsheet和OfficeQA探测任务使用小型异构任务族，而非完全交叉的方法-干预设计。例如，SkillRevise缺乏同等强度的匹配工件手术和输出锁定测量证据，而最强的验证器交换集中在Radar系列。因此，我们不声称任何观察到的机制是普遍的，也不声称一种方法优于另一种方法。

一些结论也受限于可观察性。输出锁定比较可以识别测量依赖性，但无法确定哪个验证器最能捕捉预期行为，除非有外部规范。工件取证和错误元数据控制可以排除简单的直接暴露，但无法排除隐式答案跟随策略或任何形式的污染。最后，我们的范围是冻结代理、反馈驱动且具有持久外部工件的系统；参数更新自进化、多模态技能和在线环境可能表现出不同的生命周期动态。确认性证据将需要一个共享的采样计划，该计划完全交叉实际父代和最终工件、新鲜重复、受控变体和校准验证器。

结论

我们研究基准收益是否来自自我进化的持久技能，是否与保留的行为改善相对应，以及独立地，哪些生命周期机制产生了这些收益。变体比较显示了一个实质上的技能进化器在迁移方面具有初步优势，包括当所有方法都提供工件时。执行轨迹将成功案例连接到可重用的诊断步骤和任务约束的保留。我们的审计表明，修订后的技能工件可以产生实际的行为变化，但在新的部署下，选定的收益可能会消失，迄今为止建立的迁移仍然有限。反馈、优化器变化、验证选择、工件内容、执行变异性和验证器选择都可以塑造观察到的终点。

因此，只有在有匹配证据支持的最强水平下，才应报告更高的基准分数：一个选定的收益、一个保留的工件效应、有界的受控迁移、进化路径依赖性或测量依赖性。我们提供了一个生命周期审计，将每个声明与所需的干预措施和报告证据联系起来。可迁移技能学习的声明需要冻结的从父代到最终改进，该改进在有效测量下存活并经受预声明的任务变化；较弱的证据应更严格地报告。

参考文献

Amodei, D.; Olah, C.; Steinhardt, J.; Christiano, P.; Schulman, J.; 和 Mané, D. 2016. AI安全的具体问题. arXiv:1606.06565. Chen, Z.; Guo, Z.; Huang, B.; Lu, B.; Lin, J.; Zhou, Y.; 和 Zhang, W. 2026. SkillJuror: 衡量代理技能组织如何改变运行时行为. arXiv:2606.11543. Han, T.; Zhang, Y.; Song, W.; Fang, C.; Chen, Z.; Sun, Y.; 和 Hu, L. 2026. SWE-Skills-Bench: 代理技能是否真的有助于实际软件工程? arXiv:2603.15401. Huang, Z.; Xu, J.; Yang, Y.; Gong, Z.; Yang, Q.; Tian, M.; Wang, X.; Lv, C.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, X.; Chen, D.; Zheng, X.; 和 Luo, C. 2026. 从原始经验到技能消耗：模型生成代理技能的系统研究. arXiv:2605.23899. Jiang, S.; Ma, L.; Hong, Z.; Wang, K.; Lu, Z.; Wang, T.; Chen, S.; Zhang, J.; Pan, T.; Li, W.; Liang, J.; 和 Xiao, Y. 2026. SEA-Eval: 超越情景评估的自进化代理基准. arXiv:2604.08988. Li, X.; Liu, Y.; Chen, W.; You, B.; Di, Z.; He, Y.; Zheng, S.; Choe, K. W.; Sun, J.; Wang, S.; Tao, C.; Li, B.; Zhao, X.; Geng, H.; Wu, X.; Zhou, J.; Chen, X.; Xing, H.; Li, Y.; Zeng, Q.; Wang, D.; Wang, Y.; Chaim, R. B.; Jiang, P.; Shen, H.; Kong, L.; Liu, X.; Wang, R.; Liu, X.; Li, J.; Lan, X.; Lin, Y.; Ye, W.; He, J.; Li, S.; Zhang, Y.; Gao, Y.; Li, Y.; Ma, Z.; Jing, L.; Wang, T.; Li, K.; Xue, Y.; Lyu, H.; He, Y.; Tian, Y.; Wu, S.; Wang, B.; Gao, Y.; Chen, B.; Liu, L.; Cheng, S.; Bao, J.; Tong, S.; Xu, S.; Zhuo, T. Y.; Ye, T.; Qi, Q.; Li, M.; Liao, L.; Tan, Z.; Shi, C.; Tang, X.; Tankasala, S.; Yuan, B.; Qian, Y.; Tu, J.; Wang, C.; Sun, Y.; Wang, W.; Taylor, A.; Yang, Z.; Guan, C.; Dong, Z.; Zhang, X.; Dillmann, S.; chung Lee, H.; 和 Song, D. 2026. SkillsBench: 跨越多样化任务的代理技能基准测试. arXiv:2602.12670. Li, Z.; 和 Hu, Y. 2026. SkillHone: 通过持久决策历史持续进化代理技能的框架. arXiv:2606.08671.

Liu, Y.; Ji, J.; An, L.; Jaakkola, T.; Zhang, Y.; and Chang, S. 2026a. How Well Do Agentic Skills Work in the Wild: Benchmarking LLM Skill Usage in Realistic Settings. arXiv:2604.04323.

Liu, Y.; Su, Z.; Xie, L.; Zhang, Y.; Zong, Q.; Guo, J.; Xie, Z.; Ji, Y.; Yim, Y.; Luo, H.; Ren, X.; Ruan, C.; Li, H.; and Song, Y. 2026b. SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision. arXiv:2606.01139.

Oren, Y.; Meister, N.; Chatterji, N.; Ladhak, F.; and Hashimoto, T. B. 2023. Proving Test Set Contamination in Black Box Language Models. arXiv:2310.17623.

Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; Zheng, Z.; Xie, C.; and Yao, H. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. arXiv:2602.08234.

Yang, Y.; Gong, Z.; Huang, W.; Yang, Q.; Zhou, Z.; Huang, Z.; Li, Y.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, Y.; Chen, D.; Yang, X.; and Luo, C. 2026. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. arXiv:2605.23904.

Zhang, G.; Zhu, E.; Zhou, J.; Jia, C.; and Wang, H. 2026a. SkillEvolver: Skill Learning as a Meta-Skill. arXiv:2605.10500.

Zhang, H.; Fan, S.; Zou, H. P.; Chen, Y.; Wang, Z.; Zhou, J.; Li, C.; Huang, W.-C.; Yao, Y.; Zheng, K.; Liu, X.; Li, X.; and Yu, P. S. 2026b. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. arXiv:2604.01687.

Zhang, X.; Wang, G.; Cui, Y.; Li, Z.; Qiu, W.; Zhu, B.; and He, P. 2026c. Who Grades the Grader? Co-Evolving Evaluation Metrics and Skills for Self-Improving LLM Agents. arXiv:2607.12790.

Zhong, S.; Lu, Y.; Ning, J.; Wan, Y.; Feng, L.; Ao, Y.; Ribeiro, L. F. R.; Dreyer, M.; Ammirati, S.; and Xiong, C. 2026. SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks. arXiv:2604.20087.

Zhou, X.; Liu, J.; Li, L.; Rossi, R. A.; and Hu, X. 2026a. Counterfactual Trace Auditing of LLM Agent Skills. arXiv:2605.11946.

Zhou, Y.; Dong, Z.; Wang, Z.; Jin, C.; Zhao, S.; Guo, B.; Gu, D.; Zhang, L.; Zhou, M.; and Metaxas, D. N. 2026b. Evidence Over Plans: Online Trajectory Verification for Skill Distillation. arXiv:2605.09192.

Zhu, J.; Mao, K.; Guo, Y.; He, D.; Xu, S.; Gu, S.; and Yue, Y. 2026. SkillCoach: Self-Evolving Rubrics for Evaluating and Enhancing Agentic Skill-Use. arXiv:2607.01874.

Liu, Y.; Ji, J.; An, L.; Jaakkola, T.; Zhang, Y.; 和 Chang, S. 2026a. 代理技能在现实中的表现如何：在真实场景中基准测试LLM技能使用. arXiv:2604.04323. Liu, Y.; Su, Z.; Xie, L.; Zhang, Y.; Zong, Q.; Guo, J.; Xie, Z.; Ji, Y.; Yim, Y.; Luo, H.; Ren, X.; Ruan, C.; Li, H.; 和 Song, Y. 2026b. SkillRevise: 通过轨迹条件技能修订改进LLM生成的代理技能. arXiv:2606.01139. Oren, Y.; Meister, N.; Chatterji, N.; Ladhak, F.; 和 Hashimoto, T. B. 2023. 证明黑盒语言模型的测试集污染. arXiv:2310.17623. Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; Zheng, Z.; Xie, C.; 和 Yao, H. 2026. SkillRL: 通过递归技能增强强化学习进化代理. arXiv:2602.08234. Yang, Y.; Gong, Z.; Huang, W.; Yang, Q.; Zhou, Z.; Huang, Z.; Li, Y.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, Y.; Chen, D.; Yang, X.; 和 Luo, C. 2026. SkillOpt: 自进化代理技能的执行策略. arXiv:2605.23904. Zhang, G.; Zhu, E.; Zhou, J.; Jia, C.; 和 Wang, H. 2026a. SkillEvolver: 技能学习作为一种元技能. arXiv:2605.10500. Zhang, H.; Fan, S.; Zou, H. P.; Chen, Y.; Wang, Z.; Zhou, J.; Li, C.; Huang, W.-C.; Yao, Y.; Zheng, K.; Liu, X.; Li, X.; 和 Yu, P. S. 2026b. CoEvoSkills: 通过协同进化验证自进化代理技能. arXiv:2604.01687. Zhang, X.; Wang, G.; Cui, Y.; Li, Z.; Qiu, W.; Zhu, B.; 和 He, P. 2026c. 谁评估评估者？协同进化评估指标和技能以改进自提升LLM代理. arXiv:2607.12790. Zhong, S.; Lu, Y.; Ning, J.; Wan, Y.; Feng, L.; Ao, Y.; Ribeiro, L. F. R.; Dreyer, M.; Ammirati, S.; 和 Xiong, C. 2026. SkillLearnBench: 在现实世界任务上基准测试代理技能生成的持续学习方法. arXiv:2604.20087. Zhou, X.; Liu, J.; Li, L.; Rossi, R. A.; 和 Hu, X. 2026a. LLM代理技能的反事实轨迹审计. arXiv:2605.11946. Zhou, Y.; Dong, Z.; Wang, Z.; Jin, C.; Zhao, S.; Guo, B.; Gu, D.; Zhang, L.; Zhou, M.; 和 Metaxas, D. N. 2026b. 证据胜于计划：技能蒸馏的在线轨迹验证. arXiv:2605.09192. Zhu, J.; Mao, K.; Guo, Y.; He, D.; Xu, S.; Gu, S.; 和 Yue, Y. 2026. SkillCoach: 自进化评估和增强代理技能使用的评分标准. arXiv:2607.01874.